Paneles solares: clasificación y detección

Consigna

Encontrar paneles solares en imágenes aéreas sirve, por ejemplo, para que una distribuidora eléctrica sepa dónde hay generación propia o para estimar la capacidad instalada en una zona. Puede plantearse de dos maneras:

- **Clasificación:** ¿hay paneles en esta imagen, sí o no?
- **Detección:** ¿dónde está cada panel?

En este trabajo van a utilizar el mismo dataset de las dos maneras, y van a comparar qué obtienen con cada una.

Los datos

Cada grupo recibe un zip con dos carpetas:

- `images/`: imágenes aéreas de 256×256 px en PNG.
- `labels/`: un archivo `.txt` por imagen, con el mismo nombre, que contiene la etiqueta en formato YOLO.

El zip no trae una división en entrenamiento y validación.

Organización

- Trabajen en Google Colab con GPU (*Entorno de ejecución → Cambiar tipo de entorno → T4*).
- Son varios integrantes en el equispo: repártanse los experimentos para trabajar en paralelo, y pónganse de acuerdo en la división de los datos para que los resultados sean comparables.
- Guarden los resultados a medida que los van obteniendo y los pesos de los modelos. La sesión de Colab puede cortarse.

1. Exploración de los datos

Antes de entrenar, conozcan el dataset. Qué hay, cuánto hay de cada cosa, cómo son las imágenes y las cajas. Dibujen cajas sobre algunas imágenes. Anoten lo que les llame la atención: va a servir para interpretar los resultados después.

2. Clasificación

Tarea: clasificar cada imagen en *con paneles* / *sin paneles*.

Qué probar:

- Al menos dos o tres redes distintas.
- Prueben con pre-entrenamiento (pesos de ImageNet) y sin pre-entrenamiento (pesos aleatorios).
- Con pre-entrenamiento: entrenar solo la cabeza (el extractor congelado) contra entrenar también el extractor, una parte o completo.

- Cualquier otra variable que les parezca interesante: tamaño de entrada, data augmentation, cantidad de épocas, learning rate son bienvenidas.

Guía:

- Redes disponibles en Keras: `MobileNetV2`, `EfficientNetB0`, `ResNet50`, `VGG16`, entre otras. Se cargan sin la cabeza original con `include_top=False`, y se elige `weights="imagenet"` o `weights=None`, dependiendo si cargan la red preentrenada o no.
- Para congelar el extractor: `base.trainable = False`. Para el fine-tuning, primero se entrena la cabeza con el extractor congelado y después se descongela con un learning rate bajo (del orden de `1e-5`). Cada vez que cambian `trainable`, hay que volver a compilar. Ejemplo:

  # Etapa 1: extractor congelado, se entrena solo la cabeza

  base.trainable = False model.compile(optimizer=keras.optimizers.Adam(1e-3)

  loss="binary_crossentropy", metrics=["accuracy"])

  model.fit(train_ds, validation_data=val_ds, epochs=10)

  # Etapa 2: descongelar el extractor y VOLVER A COMPILAR (con learning rate bajo)

  base.trainable = True model.compile(optimizer=keras.optimizers.Adam(1e-5)

  loss="binary_crossentropy", metrics=["accuracy"])

  model.fit(train_ds, validation_data=val_ds, epochs=10)
- Si descongelan el extractor, llamen a la base con `training=False` al armar el modelo. La guía de Keras explica por qué.

Para leer:

- Keras, *Transfer learning & fine-tuning*: https://keras.io/guides/transfer_learning/
- Keras, catálogo de redes pre-entrenadas: https://keras.io/api/applications/
- scikit-learn, métricas de clasificación: https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

3. Detección

Tarea: detectar los paneles en cada imagen.

Qué probar:

- **YOLO:** al menos dos versiones, `yolov8n.pt` y `yolo11n.pt` (opcional: `yolo26n.pt`).
- **Opcional Familia R-CNN:** Faster R-CNN de torchvision.
- Entrenar toda la red contra congelar una parte.

Guía para YOLO (Ultralytics):

- Ultralytics necesita un archivo `data.yaml` con la ubicación de las imágenes de entrenamiento y validación y los nombres de las clases. `train` y `val` pueden ser carpetas o archivos `.txt` con una lista de rutas de imágenes.
- Las imágenes son de 256 px: entrenen con `imgsz=256`.

- `freeze=N` congela las primeras N capas de la red. Para saber qué hay en cada capa: `model.info(detailed=True)` o `print(model.model)`. La arquitectura de cada versión está definida en un archivo `.yaml`, donde se ve dónde termina el backbone y dónde empieza la cabeza.
- `pretrained=False` entrena desde pesos aleatorios.

Guía para Faster R-CNN (opcional):

- Hay dos versiones: `fasterrcnn_mobilenet_v3_large_fpn` (más liviana) y `fasterrcnn_resnet50_fpn`.
- Se parte de los pesos de COCO y se reemplaza la cabeza (`FastRCNNPredictor`) por una con la cantidad de clases del dataset más una, porque en torchvision la clase 0 es el fondo.
- `trainable_backbone_layers` controla cuántos bloques del extractor se entrenan, contando desde el final: 0 congela todo el extractor.
- Pasen `min_size=256, max_size=256`. Si no, torchvision agranda las imágenes a 800 px y el entrenamiento se vuelve mucho más lento.
- El archivo `ayuda_rcnn.py` trae armadas las piezas más trabajosas: el dataset que lee las etiquetas YOLO, el bucle de entrenamiento y el cálculo de mAP. Pueden usarlo tal cual, modificarlo o escribir el suyo a partir del tutorial oficial.

Para leer:

- Ultralytics, modo de entrenamiento y todos sus parámetros (`freeze`, `imgsz`, `pretrained`, `epochs`…): https://docs.ultralytics.com/modes/train/
- Ultralytics, formato de los datasets de detección y del `data.yaml`: https://docs.ultralytics.com/datasets/detect/
- Ultralytics, entrenar con capas congeladas. Está escrito para YOLOv5, pero la idea es la misma: https://docs.ultralytics.com/yolov5/tutorials/transfer_learning_with_frozen_layers
- Arquitecturas de YOLOv8 y YOLO11, capa por capa:
  - https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/v8/yolov8.yaml
  - https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/11/yolo11.yaml
- PyTorch, tutorial oficial de fine-tuning de Faster R-CNN: https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html
- torchvision, Faster R-CNN MobileNet y sus parámetros: https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.detection.fasterrcnn_mobilenet_v3_large_fpn.html

4. Métricas y comparación

- **Clasificación:** matriz de confusión, precision, recall y F1, además de la accuracy.
- **Detección:** mAP50, mAP50-95 y la curva precisión-recall.
- **Detector como clasificador:** si el detector encuentra al menos un panel en una imagen, esa imagen "tiene paneles". Calculen las mismas métricas de clasificación sobre las mismas imágenes de validación y compárenlas con las de sus clasificadores.

Para leer:

- Ultralytics, guía de métricas de detección (IoU, precision, recall, mAP, curvas): https://docs.ultralytics.com/guides/yolo-performance-metrics/
- torchmetrics, `MeanAveragePrecision`: https://lightning.ai/docs/torchmetrics/stable/detection/mean_average_precision.html

5. Entrega

Informe, comparaciones y conclusiones. Es importante que entiendan las particularidades de los modelos que utilizaron: arquitectura, hiperparámetros, estrategias de entrenamiento.